Operating Systems (R20A0504) - Study Notes

Course Objectives:

1. Understand the fundamental concepts and techniques of Operating Systems.
2. Study the concepts of Linux shell and scheduling.
3. Understand the concepts in deadlocks and process management.
4. Understand the concepts in memory management and IPC mechanisms.
5. Study the concept of file system concepts and sockets.

UNIT – I: Operating System - Introduction

1. Introduction to Operating Systems

- **Definition:** An operating system (OS) acts as a manager for computer resources (hardware and software), allocating them to programs and users as needed. It serves as an interface between the user and the machine.
- **Core Functions:**
    - Resource Management (CPU, memory, files, I/O devices).
    - Provides an environment for other programs to execute.
    - Manages hardware and coordinates its use among various applications and users.
- **Computer System Components:**
    - **Computer Hardware:** CPU, memory, I/O devices (provides basic computing resources).
    - **Application Programs:** Solve user computing problems (e.g., spreadsheets, web browsers).
    - **System Programs:** Compilers, loaders, editors, OS itself.
    - **Operating System:** Controls hardware, coordinates resource usage.
- **Two Primary Purposes of OS:**

1. Controls the allocation and use of computing system resources among users and tasks.

2. Provides an interface between hardware and programmers, simplifying application development and debugging.

2. Views of Operating System

- **User View:** Focuses on the interface used by the user. Designed for ease of use, with some attention to performance, but often little to resource utilization.
- **System View:** Views the OS as a resource allocator. Manages hardware and software resources efficiently, deciding between conflicting requests, and controlling program execution.

3. Types of Operating Systems

- **Simple Batch System:**
    - Similar jobs are grouped into batches and executed sequentially.
    - **Advantage:** Reduced execution time for similar jobs (e.g., loading a compiler once for a batch of C++ programs).
    - **Disadvantage:** Low CPU utilization due to high overhead of loading/unloading batches; manual intervention required between batches.
- **Multiprogramming:**
    - Multiple programs reside in memory simultaneously, sharing a single processor.
    - Increases CPU utilization by ensuring the CPU always has a job to execute.
    - **Activities:** OS keeps several jobs in memory, picks one to execute, monitors states of active programs.
    - **Advantages:** High and efficient CPU utilization; users perceive simultaneous execution.
    - **Disadvantages:** Requires CPU scheduling and memory management.
- **Time-Sharing Operating System (Multitasking):**
    - Multiple processes execute concurrently by dividing CPU time into small units called "time quanta."
    - Each process gets a chance to execute for its time quantum before being switched out.
    - **Advantages:** Equal opportunity for each process; high CPU utilization.

    - **Disadvantages:** Processes with higher priority may not get immediate execution due to equal time allocation.
- **Personal Computer (PC) Operating System:**
    - Provides a user-friendly interface for a single user.
    - Widely used for word processing, spreadsheets, internet access (e.g., Windows, macOS, Android).
- **Parallel Processing:**
    - Multiple processors work simultaneously on a task divided into sub-parts.
    - Completes jobs in the shortest possible time.
    - All processors run the same OS.
    - Processors are tightly coupled, share secondary storage and user terminals.
    - **Advantages:** Saves time and money; solves larger problems efficiently; utilizes hardware better.
    - **Disadvantages:** Difficult communication and synchronization between sub-tasks; algorithms must be designed for parallel execution; requires skilled programmers.
- **Distributed Operating System:**
    - Multiple autonomous computers communicate over a network.
    - Each system has its own memory and CPU (loosely coupled).
    - **Advantages:** Failure of one system doesn't affect others; fast data exchange; high computation speed and durability; scalable; reduced load on host computers.
    - **Disadvantages:** Failure of the main network stops communication; language for distributed systems is not well-defined; expensive.
- **Real-Time Operating System (RTOS):**
    - Designed for applications requiring data processing within a fixed, small duration.
    - Requires quick input and immediate response.
    - **Types:**

        - **Hard Real-Time Systems:** Timing is critical; even a fraction of a second delay can be disastrous (e.g., airbags, missile launch systems). Lack virtual memory.
        - **Soft Real-Time Systems:** Timing is less critical; failure to meet deadlines degrades performance but isn't catastrophic (e.g., video surveillance, video players).
    - **Advantages:** High and quick output; fast task shifting; prioritizes current applications; suitable for embedded systems; error-free; efficient memory allocation.
    - **Disadvantages:** Fewer tasks can run simultaneously; complex algorithms required; specific drivers and interrupt signals needed; can be expensive.

4. Operating System Services

- **Program Execution:** Loading and running programs, handling normal/abnormal termination.
- **I/O Operations:** Managing communication between user and device drivers; providing access to I/O devices.
- **File System Manipulation:** Creating, deleting, reading, writing files and directories; managing storage media (magnetic tape, disk, optical disk).
- **Communication:** Managing communication between processes, especially in distributed systems via networks.
- **Error Detection:** Constantly monitoring the system for errors (CPU, I/O, memory) and taking appropriate actions.
- **Resource Allocation:** Managing and allocating resources (CPU, memory, devices) to multiple users/jobs.
- **Protection:** Securing processes and controlling access to information and resources.

5. Operating System Structures

- **Simple Structure:**
    - No well-defined structure; small, simple systems (e.g., MS-DOS).
    - Application programs can access basic I/O routines.

    - **Disadvantage:** System crash if a user program fails; complicated structure with no clear boundaries.
- **Layered Structure:**
    - OS is divided into layers (hardware at layer 0, user interface at layer N).
    - Each layer uses functions of lower-level layers only.
    - **Advantage:** Simplifies debugging; easier to enhance.
    - **Disadvantage:** Degraded application performance due to overhead; requires careful planning of layers. (e.g., UNIX).

6. Introduction to Linux Operating System

- **Definition:** A Unix-like, free and open-source operating system.
- **Kernel:** Linux kernel, first released in 1991 by Linus Torvalds.
- **Portability:** Runs on a wide variety of hardware platforms.
- **Usage:** Dominant on servers, supercomputers, and embedded systems (including Android).
- **Basic Features:**
    - **Portable:** Works on different hardware.
    - **Open Source:** Freely available source code, community-driven development.
    - **Multi-User:** Multiple users can access system resources simultaneously.
    - **Multiprogramming:** Multiple applications can run concurrently.
    - **Hierarchical File System:** Standard file structure for organizing files.
    - **Shell:** Interpreter program for executing commands.
    - **Security:** User authentication (passwords), controlled access, encryption.
- **Advantages:**
    - **Low Cost:** Free licenses and software.
    - **Stability:** High uptime, no frequent reboots needed.
    - **Performance:** Persistent high performance, handles many users.
    - **Network Friendliness:** Strong network support.

        - **Flexibility:** Usable for servers, desktops, embedded systems.
        - **Compatibility:** Runs common UNIX software and file formats.
        - **Choice:** Numerous distributions available.
        - **Fast and Easy Installation:** User-friendly setup programs.
        - **Full Use of Hard Disk:** Works well even when the disk is nearly full.
        - **Multi-tasking:** Handles many tasks simultaneously.
        - **Security:** Robust security features.
        - **Open Source:** Source code available for modification.
    - **Linux System Architecture:**
        - **Hardware Layer:** Peripheral devices (RAM, HDD, CPU).
        - **Kernel:** Core OS component, interacts directly with hardware.
        - **Shell:** Interface to the kernel, executes user commands.
        - **Utilities:** Programs providing OS functionalities.

7. Linux File System

- **Structure:** Hierarchical, organized under a root directory (**/**).
- **Key Directories:**
    - **/**: Root directory.
    - **/bin**: User binaries (essential commands for all users).
    - **/sbin**: System binaries (commands for system administration).
    - **/etc**: Configuration files.
    - **/dev**: Device files.
    - **/proc**: Process information (virtual filesystem).
    - **/var**: Variable files (logs, databases, mail).
    - **/tmp**: Temporary files.
    - **/usr**: User programs, binaries, libraries, documentation.
    - **/home**: Home directories for users.

- **/boot**: Boot loader files.
- **/lib**: System libraries.
- **/opt**: Optional add-on applications.
- **/mnt**: Temporary mount directory.
- **/media**: Mount directory for removable media.
- **/srv**: Service data.

- **Types of Files:**
  - **Regular Files:** Text, images, binaries.
  - **Directories:** Files that store lists of other files.
  - **Special Files:** Represent physical devices (e.g., printers).
- **File Handling Utilities:**
  - **ls**: List files and directories.
  - **touch**: Create a new file.
  - **cat**: Display file contents.
  - **cp**: Copy a file.
  - **mv**: Move or rename a file.
  - **rm**: Delete a file.

8. Linux Utilities

- **Process Utilities:**
  - **Process:** A program in execution.
  - **Process ID (PID):** Unique 5-digit number for each process.
  - **Foreground Process:** Receives input from keyboard, output to screen; blocks prompt until completion.
  - **Background Process:** Runs without keyboard input; prompt remains available. Started with **&**.
  - **ps**: Display running processes.
  - **ps -f**: Full process status.

    - **pidof**: Find process IDs by name.
    - **Ctrl + c**: Interrupt a foreground process.
    - **kill**: Terminate a process (using PID).
- **Networking Commands:**
    - **ping**: Check if a remote system is running/connected.
    - **host**: Obtain network address information.
    - **finger**: Get information about network users.
    - **traceroute**: Track the route to a host.
    - **netstat**: Check port status and network statistics.
    - **tracepath**: Similar to traceroute, simpler options.
    - **dig**: Query DNS related information.
    - **hostname**: Display or set the system's hostname.
    - **route**: Display or modify the routing table.
    - **nslookup**: Query DNS servers.
- **Filters:** Programs that process text input and produce formatted output.
    - **cat**: Displays file content.
    - **head**: Displays the first N lines of a file.
    - **tail**: Displays the last N lines of a file.
    - **sort**: Sorts lines alphabetically or numerically.
    - **uniq**: Removes duplicate lines (works best on sorted data).
    - **wc**: Counts lines, words, and characters.
    - **grep**: Searches for patterns in text.
    - **tac**: Reverses the order of lines in a file.
    - **sed**: Stream editor for text transformations.
    - **nl**: Numbers lines of text.

UNIT – II: Process & CPU Scheduling

1. Shell Programming

- **Shell:** An interface between the user and the UNIX/Linux system; a command-line interpreter and a programming language.
- **Common Shells:** Bourne Shell (**sh**), C Shell (**csh**), Korn Shell (**ksh**), Bourne Again Shell (**bash**).
- **Shell Responsibilities:**
    1. **Program Execution:** Analyzes command lines and executes programs.
    2. **Variable and Filename Substitution:** Uses variables and wildcards (*****, **?**, **[]**) for dynamic commands.
    3. **I/O Redirection:** Uses **<**, **>**, **>>** to redirect input and output.
    4. **Pipeline Hookup:** Uses **|** to connect the output of one command to the input of another.
    5. **Environment Control:** Customizes user environment (home directory, prompt, search paths).
    6. **Interpreted Programming Language:** Executes commands line by line.
- **Pipes and Redirection:**
    - **>**: Redirects output to a file (overwrites).
    - **>>**: Appends output to a file.
    - **<**: Redirects input from a file.
    - **|**: Pipes the output of one command to the input of another.
- **Here Documents:** Pass input to a command from a shell script using **<< LEADER**.
- **Shell Scripts:**
    - A file containing a sequence of commands.
    - Starts with **#!/bin/sh** (shebang line).
    - Made executable using **chmod +x script_name.sh**.
    - Executed by typing **./script_name.sh**.

- **Shell Metacharacters:** Special characters with specific meanings (file substitution, I/O redirection, process execution, quoting, etc.).
- **Shell Variables:**
    - **User-defined variables: variable=value** (e.g., **x=10**). Case-sensitive.
    - **Environment Variables:** System-defined, usually capitalized (e.g., **$HOME**, **$PATH**, **$PS1**).
    - **set**: Displays all defined variables.
    - **unset**: Removes a variable.
- **Command Substitution: $(command)** or **`command`** captures the output of a command.
- **read command:** Takes input from the user and stores it in variables.
- **printf command:** Prints formatted output.
- **Exit Status: $?** stores the exit status of the last command (0 for success, non-zero for failure).
- **exit command:** Terminates a script prematurely.
- **: (colon) command:** A null command, often used for comments in older scripts.
- **expr command:** Evaluates expressions.
- **export command:** Makes a variable known to sub-shells.
- **eval command:** Scans the command line twice before execution.
- **shift command:** Left-shifts positional parameters (**$1**, **$2**, etc.).
- **Quoting:**
    - Single quotes (**'...'**): Prevent all interpretation.
    - Double quotes (**"..."**): Allow variable expansion and command substitution but prevent word splitting and globbing.
    - Backslash (****): Escapes the next character.
- **test or [] command:** Used for conditional checks (string comparison, arithmetic comparison, file conditions).
- **Control Structures:**

        - **if statement: if condition then statements fi**.
        - **elif (else if): if ... elif ... else ... fi**.
        - **for loop: for variable in values do statements done**.
        - **while loop: while condition do statements done**.
        - **until loop: until condition do statements done**.
        - **case statement: case variable in pattern1) statements;; pattern2) statements;; *) statements;; esac**.
- **Parameter Expansion:** Using **{}** around variables (e.g., **${i}_tmp**) for clarity or to avoid ambiguity.

2. Process Concepts and Scheduling

- **Process:** A program in execution. A dynamic entity.
- **Differences between Process and Program:** | Feature | Process | Program | | :---------- | :---------- | :---------- | | **Nature** | Dynamic object | Static object | | **Memory** | Loaded into main memory | Loaded into secondary storage | | **Execution** | Sequence of instruction execution | Sequence of instructions | | **Time Span** | Limited | Unlimited | | **Entity** | Active | Passive |
- **Process States:**
    - **New:** Process is being created.
    - **Ready:** Process is waiting to be assigned to a processor.
    - **Running:** Process is being executed.
    - **Waiting:** Process is waiting for some event to occur (e.g., I/O completion).
    - **Terminated:** Process has finished execution.
- **Process State Transitions:**
    - New -> Ready (OS prepares process)
    - Ready -> Running (OS selects from ready queue)
    - Running -> Terminated (Execution complete or OS termination)
    - Running -> Ready (Time slice expired, interrupt)
    - Running -> Waiting (Event/I/O required)

    - Waiting -> Ready (Event completed)
- **Process Control Block (PCB) / Task Control Block (TCB):**
    - Contains information about a specific process.
    - **Fields:** Process State, Program Counter, CPU Registers, CPU Scheduling Info, Memory Management Info, Accounting Info, I/O Status Info.
- **Threads:**
    - A lightweight process within a process.
    - Each thread has a program counter, registers, and stack.
    - **Thread States:** Born, Ready, Running, Sleep, Dead.
    - **Multithreading:** A process divided into multiple threads executing concurrently. Improves performance if multiple CPUs are available.
- **Differences between Process and Thread:** | Feature | Process | Thread | | :---------- | :---------- | :---------- | | **Creation Time**| More time | Less time | | **Termination** | More time | Less time | | **Execution** | Slower | Faster | | **Switching** | More time | Less time | | **Communication**| Difficult | Easy | | **Memory Share** | Cannot share same memory area | Can share same memory area | | **Coupling** | Loosely coupled | Tightly coupled | | **Resources** | Requires more resources | Requires fewer resources |

3. CPU Scheduling

- **Purpose:** To maximize CPU utilization and provide acceptable response times by switching the CPU between jobs.
- **Scheduling Objectives:** Maximize throughput, maximize acceptable response times, be predictable, balance resource use, avoid indefinite postponement, enforce priorities.
- **Scheduling Queues:**
    - **Job Queue:** All processes in the system.
    - **Ready Queue:** Processes in main memory ready for execution.
    - **Device Queue:** Processes waiting for a particular I/O device.
- **Schedulers:**

    - **Long-Term Scheduler (Job Scheduler):** Selects jobs from the job pool to load into main memory. Controls the degree of multiprogramming.
    - **Short-Term Scheduler (CPU Scheduler):** Selects a process from the ready queue and allocates it to the CPU.
    - **Medium-Term Scheduler:** Removes processes from memory (swaps out) to reduce multiprogramming degree, and later swaps them back in.
- **Context Switch:** The process of saving the state of a running process and restoring the state of another process to allow the CPU to switch between them.
- **Scheduling Types:**
    - **Non-Preemptive Scheduling:** CPU is assigned to a process until it completes or voluntarily releases it.
    - **Preemptive Scheduling:** CPU can be taken away from a running process if a higher-priority process becomes ready.
- **Dispatcher:** The module that gives control of the CPU to the process selected by the short-term scheduler.
- **Dispatcher Latency:** The time taken by the dispatcher to stop one process and start another.

4. Scheduling Criteria

- **Throughput:** Number of jobs completed per unit time.
- **Turnaround Time:** Time from submission to completion.
- **Waiting Time:** Time spent waiting in the ready queue.
- **Response Time:** Time from submission to the first response.
- **CPU Utilization:** Percentage of time the CPU is busy.

5. CPU Scheduling Algorithms

- **First-Come, First-Served (FCFS):**
    - Non-preemptive. Processes are executed in the order they arrive.
    - **Advantage:** Easy to implement, simple.
    - **Disadvantage:** High average waiting time; convoy effect (short processes get stuck behind long ones).

- **Shortest Job First (SJF):**
    - Non-preemptive. Selects the process with the smallest CPU burst time.
    - **Advantage:** Least average waiting time, turnaround time, and response time.
    - **Disadvantage:** Difficult to predict the next CPU burst time; potential for starvation of long jobs (aging can help).
- **Shortest Remaining Time First (SRTF):**
    - Preemptive version of SJF. Selects the process with the shortest remaining CPU burst time.
    - If a new process arrives with a shorter remaining time than the currently executing process, it preempts the current one.
- **Round Robin (RR):**
    - Designed for time-sharing systems.
    - Preemptive. Each process gets a small time quantum (time slice).
    - Uses a circular ready queue.
    - **Advantage:** Fair, provides good response time.
    - **Disadvantage:** High context switching overhead if time quantum is too small.
- **Priority Scheduling:**
    - Each process is assigned a priority. The CPU is allocated to the process with the highest priority.
    - Can be preemptive or non-preemptive.
    - **Disadvantage:** Starvation of low-priority processes.
- **Multiple-Processor Scheduling:**
    - Scheduling becomes more complex with multiple CPUs.
    - **Approaches:**
        - **Asymmetric Multiprocessing:** One master processor controls all activities.
        - **Symmetric Multiprocessing (SMP):** Each processor schedules its own jobs.

- **Processor Affinity:** Attempting to keep a process running on the same processor to benefit from cache memory.
    - **Soft Affinity:** System attempts to keep processes on the same processor but makes no guarantees.
    - **Hard Affinity:** Process specifies it must not be moved between processors.
- **Load Balancing:** Distributing processes evenly across processors to prevent overload.
    - **Push Migration:** A separate process moves processes from overloaded to less loaded processors.
    - **Pull Migration:** Idle processors take processes from other processors' ready queues.

UNIT – III: Deadlocks & Process Management

1. Deadlocks

- **System Model:**
    - A system has a finite number of resources partitioned into types.
    - Processes request, use, and release resources.
    - A process must request a resource before using it and release it after use.
    - Resource requests can be granted immediately or the process must wait.
- **Deadlock Definition:** A situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process in the set.
- **Necessary Conditions for Deadlock:**

1. **Mutual Exclusion:** Only one process can use a resource at a time.

2. **Hold and Wait:** A process holds at least one resource and waits for additional resources held by other processes.

3. **No Preemption:** Resources cannot be preempted; they can only be released voluntarily.

4. **Circular Wait:** A cycle of processes exists where each process waits for a resource held by the next process in the cycle.

- **Resource Allocation Graph (RAG):**
    - Vertices: Processes (P) and Resource Types (R).
    - Edges:
        - Request Edge (**Pi -> Rj**): Process **Pi** requests resource **Rj**.
        - Assignment Edge (**Rj -> Pi**): Resource **Rj** is allocated to process **Pi**.
    - **Cycle in RAG:** A cycle is a necessary condition for deadlock. If each resource type has only one instance, a cycle is also a sufficient condition.
- **Methods for Handling Deadlocks:**

1. **Deadlock Prevention:** Ensure at least one of the four necessary conditions cannot hold.
    - **Preventing Hold and Wait:** Require processes to request all resources at once or release all held resources before requesting new ones. (Low resource utilization, starvation possible).
    - **Preventing No Preemption:** If a process holding resources requests another that cannot be immediately allocated, preempt all held resources.
    - **Preventing Circular Wait:** Impose a total ordering of resource types and require processes to request resources in increasing order.
2. **Deadlock Avoidance:** Dynamically examine the resource allocation state to ensure the system never enters an unsafe state.
    - Requires additional a priori information (e.g., maximum resource needs per process).
    - **Safe State:** A state where there exists a sequence of processes such that all can complete execution. If a system is safe, no deadlocks will occur.
    - **Banker's Algorithm:** A deadlock avoidance algorithm for multiple instances of resources. It uses **Available**, **Max**, **Allocation**, and **Need** matrices.
3. **Deadlock Detection:** Allow the system to enter a deadlock state and then detect it.

- **Single Instance of Each Resource Type:** Maintain a wait-for graph and search for cycles.
- **Multiple Instances of Resource Types:** Use a detection algorithm similar to the Banker's algorithm (**Available**, **Allocation**, **Request** matrices).

4. **Deadlock Recovery:** Once detected, break the deadlock.
    - **Process Termination:** Abort all deadlocked processes or one at a time until the cycle is broken.
    - **Resource Preemption:** Select a victim process, preempt its resources, and roll back the process to a safe state.
    - **Starvation:** A process may be repeatedly selected as a victim.

2. Process Management and Synchronization

- **Critical Section Problem:**
    - A segment of code where a process may access shared resources.
    - **Requirement:** Only one process can execute its critical section at any given time (Mutual Exclusion).
    - **Other Requirements:** Progress (if no process is in CS, any process wanting to enter can), Bounded Waiting (limits on how many times others can enter CS before a process gets its turn).
- **Synchronization Hardware:**
    - **Disabling Interrupts:** Effective on uniprocessors but inefficient and problematic on multiprocessors.
    - **Test-and-Set Instruction:** Atomically tests and sets a boolean variable.
    - **Exchange Instruction:** Atomically exchanges the contents of two memory locations.
    - **Disadvantages of Hardware Solutions:** Busy-waiting (spinlock), starvation, deadlock.
- **Semaphores:**
    - A synchronization tool (integer variable) accessed via atomic **wait()** (P) and **signal()** (V) operations.

    - **Counting Semaphore:** Value can range over an unrestricted domain.
    - **Binary Semaphore:** Value ranges between 0 and 1 (acts like a mutex).
    - **Busy Waiting (Spinlock):** Process repeatedly checks the semaphore value. Useful in multiprocessor systems for short critical sections.
    - **Blocking Semaphores:** Process blocks if it cannot acquire the semaphore, avoiding busy-waiting.
- **Classical Problems of Synchronization:**
    - **Producer-Consumer Problem (Bounded-Buffer):** Processes produce items and place them in a buffer, while other processes consume items from the buffer. Solved using **mutex**, **empty**, and **full** semaphores.
    - **Dining Philosophers Problem:** Five philosophers alternate between thinking and eating. Each needs two forks to eat. Solved using semaphores to manage forks and prevent deadlock/starvation (e.g., asymmetric solution, limiting philosophers at the table).
    - **Readers-Writers Problem:** Multiple processes need to access a shared data object. Readers can access concurrently, but writers need exclusive access. Solved using semaphores to manage reader count and writer access. Potential for writer starvation.
    - **Sleeping Barber Problem:** A barber shop with one barber, one barber chair, and waiting chairs. Barber sleeps if no customers; customers wake the barber or wait. Solved using semaphores (**customers**, **barber**, **mutex**).
- **Critical Regions:** High-level synchronization construct that guards shared data. Reduces synchronization errors but doesn't eliminate them.
- **Monitors:**
    - A high-level synchronization construct that encapsulates shared data, procedures operating on it, and condition variables.
    - Only one process can be active within a monitor at a time.
    - **Operations: wait()** and **signal()** on condition variables.
    - **Advantages:** Easier to program than semaphores, reduces scope of errors.
- **Message Passing:**

- Used for inter-process communication, especially in distributed systems.
- Processes communicate by sending and receiving messages.
- **Primitives: send(destination, message)** and **receive(source, message)**.
- **Types:** Sockets, pipes, message queues.

UNIT – IV: Interprocess Communication & Memory Management

1. Interprocess Communication (IPC) Mechanisms

- **IPC:** OS-supported mechanisms for interaction, coordination, and communication among processes.
- **Forms of IPC:**
    - **Message Passing:** Processes exchange messages via channels managed by the OS.
        - **Pipes:** Carry a byte stream between two related processes (e.g., output of one to input of another).
        - **Message Queues:** Carry discrete "messages" among processes, often with priorities. (Sys-V and POSIX APIs).
        - **Sockets:** Provide endpoints for communication, used for network communication (different machines) or local communication.
          Uses **send()** and **recv()** system calls.
    - **Shared Memory:** Processes read and write to a common memory region established by the OS.
        - **Advantages:** High performance, fewer system calls after setup.
        - **Disadvantages:** Requires explicit synchronization (mutexes, semaphores) to manage concurrent access.
- **System Calls for IPC:**
    - **Pipes: pipe()**
    - **Message Queues: msgget()**, **msgsnd()**, **msgrcv()**, **msgctl()** (Sys-V); **mq_open()**, **mq_send()**, **mq_receive()**, **mq_close()** (POSIX).

    - **Shared Memory: shmget()**, **shmat()**, **shmdt()**, **shmctl()** (Sys-V); **shm_open()**, **mmap()**, **munmap()**, **shm_unlink()** (POSIX).
    - **Sockets: socket()**, **bind()**, **listen()**, **accept()**, **connect()**, **send()**, **recv()**, **close()**.
- **Synchronization Primitives for IPC:**
    - **Mutexes:** Ensure mutual exclusion for accessing shared resources.
    - **Semaphores:** General synchronization construct (counting or binary).
    - **Condition Variables:** Used with mutexes to signal events and allow processes to wait.
    - **Spinlocks:** Low-level synchronization using hardware atomic instructions; involves busy-waiting.
    - **Reader-Writer Locks:** Allow multiple readers or a single writer.
    - **Monitors:** High-level construct encapsulating data and synchronization.
- **Hardware Support for Synchronization:** Atomic instructions (Test-and-Set, Compare-and-Swap) are crucial for implementing efficient synchronization primitives.

2. Memory Management and Virtual Memory

- **Background:** Efficient memory management is critical for OS performance, especially with multitasking.
- **Basic Hardware:** CPU accesses registers and main memory. I/O devices interact via memory-mapped I/O or I/O ports. Caches speed up memory access. Hardware enforces protection to prevent processes from accessing unauthorized memory.
- **Address Binding:** Mapping symbolic names to physical memory addresses.
    - **Compile Time:** Absolute code generated; requires recompilation if load address changes.
    - **Load Time:** Relocatable code generated; requires reloading if load address changes.
    - **Execution Time:** Binding delayed until execution; requires hardware support (relocation register). Most modern OSes use this.
- **Logical vs. Physical Address Space:**
    - **Logical Address:** Generated by the CPU.

        - **Physical Address:** Address seen by the memory hardware.
        - **Virtual Address:** Logical address when execution-time binding is used.
        - **Memory Management Unit (MMU):** Hardware that maps logical to physical addresses (e.g., using a relocation register).
- **Dynamic Loading:** Routines are loaded into memory only when called, saving memory.
- **Dynamic Linking & Shared Libraries:**
    - Library code is linked at run time, not compile time.
    - Saves disk space and memory if libraries are reentrant (can be shared).
    - Allows easier updates of libraries.
- **Swapping:** Moving processes between main memory and a backing store (fast disk) to free up memory. Slow process; requires execution-time binding for flexibility. Most modern OSes use paging instead.
- **Contiguous Memory Allocation:**
    - Each process occupies a single contiguous block of physical memory.
    - **Memory Protection:** Achieved using base and limit registers.
    - **Allocation Strategies:**
        - **Fixed Partitions:** Memory divided into fixed-size partitions (no longer used).
        - **Variable Partitions:** Free memory is a list of holes.
            - **First Fit:** Allocate the first hole that is large enough.
            - **Best Fit:** Allocate the smallest hole that is large enough.
            - **Worst Fit:** Allocate the largest hole.
    - **Fragmentation:**
        - **External Fragmentation:** Total free memory is sufficient, but not contiguous.
        - **Internal Fragmentation:** Allocated memory is larger than needed (e.g., within a fixed-size partition or page).

    - **Compaction:** Moving processes to consolidate free memory (requires relocatable programs).
- **Segmentation:**
    - Memory is divided into logical segments (code, data, stack, heap).
    - Addresses are (segment number, offset).
    - **Hardware:** Segment table maps segment numbers to base addresses and limits.
    - **Advantage:** Supports programmer's view of memory.
- **Paging:**
    - Physical memory divided into fixed-size **frames**.
    - Logical memory divided into fixed-size **pages** (same size as frames).
    - **Page Table:** Maps page numbers to frame numbers.
    - **Logical Address:** (Page Number, Offset).
    - **Physical Address:** (Frame Number, Offset).
    - **Advantage:** Eliminates external fragmentation; allows non-contiguous allocation.
    - **Disadvantage:** Internal fragmentation; page table overhead.
    - **Hardware Support:** Page-table base register (PTBR), Translation Look-aside Buffer (TLB) for faster lookups.
    - **Protection:** Valid/invalid bits and access rights in page table entries.
    - **Shared Pages:** Allows multiple processes to share code pages.
- **Structure of the Page Table:**
    - **Hierarchical Paging:** Two-level or multi-level paging to handle large address spaces efficiently.
    - **Hashed Page Tables:** Uses hashing to manage page table entries.
    - **Inverted Page Tables:** One entry per frame, mapping physical frames to logical pages.
- **Virtual Memory:**
    - Allows processes to have a logical address space larger than physical memory.

- **Demand Paging:** Pages are loaded into memory only when needed (on page fault).
- **Page Fault:** Occurs when a requested page is not in physical memory. Handled by the OS: check validity, find free frame, load page from disk, update page table, restart instruction.
- **Pure Demand Paging:** No pages are loaded until a page fault occurs.
- **Performance:** Page faults significantly increase access time. Low page fault rate is crucial.
- **Page Replacement:** When no free frames are available, an algorithm selects a victim page to be swapped out.
    - **Algorithms:**
        - **FIFO (First-In, First-Out):** Replaces the oldest page. Can suffer from Belady's Anomaly.
        - **Optimal (OPT/MIN):** Replaces the page that will not be used for the longest time in the future (unimplementable benchmark).
        - **LRU (Least Recently Used):** Replaces the page that has not been used for the longest time. Approximates OPT. Requires hardware support (counters or stack).
        - **LRU Approximation:** Uses reference bits (e.g., Second-Chance/Clock algorithm, Enhanced Second-Chance).
        - **Counting-Based:** LFU (Least Frequently Used), MFU (Most Frequently Used).
- **Frame Allocation:** How many frames are allocated to each process (equal, proportional, local, global).
- **Thrashing:** A process spends more time paging than executing, leading to severe performance degradation. Occurs when a process doesn't have enough frames to hold its working set.
- **Working-Set Model:** Tracks the set of pages a process actively uses (its working set) to determine adequate frame allocation.
- **Page-Fault Frequency (PFF):** Monitors page fault rate to adjust frame allocation.

UNIT – V: File System Interface and Operations

1. File System Interface

- **File Attributes:** Name, identifier, type, location, size, protection, time/date, user ID.
- **File Operations:** Create, write, read, reposition, delete, truncate.
- **File Types:** Determined by OS (e.g., Windows extensions) or internal structure (e.g., UNIX magic numbers, Macintosh forks).
- **File Structure:**
    - **Physical Blocks:** Files are stored in fixed-size blocks on disk.
    - **Logical Units:** Files are accessed as sequences of bytes or records.
    - **Packing:** How many logical units fit into a physical block.
- **Access Methods:**
    - **Sequential Access:** Read/write records in order (like tape).
    - **Direct Access:** Read/write records at any position (random access).
    - **Indexed Access:** Uses an index structure for faster access.

2. Directory Structure

- **Storage Structure:** Disk can be partitioned into multiple file systems.
- **Directory Overview:** Operations include search, create, delete, list, rename, traverse.
- **Directory Structures:**
    - **Single-Level:** All files in one directory; requires unique names.
    - **Two-Level:** User directories managed by a master file directory.
    - **Tree-Structured:** Hierarchical structure with current directory and pathnames (absolute/relative).
    - **Acyclic-Graph Directories:** Allows file sharing via links (hard links, symbolic links).
    - **General Graph Directory:** Allows cycles, requiring garbage collection.
- **File System Mounting:** Combining multiple file systems into a single directory tree.

3. Protection

- **Goals:** Reliability (prevent accidental damage) and Protection (prevent malicious access).

- **Types of Access:** Read, Write, Execute, Append, Delete, List.
- **Access Control:**
    - **Access Control Lists (ACLs):** Specify permissions for specific users/groups.
    - **UNIX Permissions:** Owner, Group, Others (Read, Write, Execute bits). Special bits (SUID, SGID, Sticky bit).
- **Other Protection Approaches:** Passwords, encryption.

4. File System Structure

- **Layered Design:** Physical devices -> I/O Control (Device Drivers) -> Basic File System -> File Organization Module -> Logical File System.
- **Common File Systems:** UFS, FFS, FAT, FAT32, NTFS, ISO 9660, ext2, ext3.

5. File System Implementation

- **On-Disk Structures:** Boot control block, volume control block, directory structure, File Control Block (FCB)/inode.
- **In-Memory Structures:** Mount table, directory cache, system-wide open file table, per-process open file table.
- **Partitions and Mounting:** Dividing disks into partitions, formatting them with file systems, and mounting them into the directory tree.
- **Virtual File Systems (VFS):** Provides a common interface to different file system types using objects like inodes, file objects, superblocks, and dentries.

6. Allocation Methods

- **Contiguous Allocation:**
    - File blocks are stored contiguously on disk.
    - **Advantages:** Fast sequential and random access.
    - **Disadvantages:** External fragmentation, difficulty with growing files.
    - **Extent-based allocation:** Allocates contiguous blocks in larger chunks (extents).
- **Linked Allocation:**
    - File blocks are linked together using pointers.
    - **Advantages:** No external fragmentation, dynamic file growth.

        - **Disadvantages:** Slow random access, reliability issues if pointers are lost, overhead for pointers.
        - **File Allocation Table (FAT):** A variation where all links are stored in a separate table.
    - **Indexed Allocation:**
        - Each file has an index block containing pointers to its data blocks.
        - **Advantages:** Supports random access efficiently, dynamic file growth.
        - **Disadvantages:** Overhead for index blocks, potential for wasted space if index blocks are large.
        - **Schemes:** Linked, Multi-Level Index, Combined (e.g., UNIX inodes).

7. Free-Space Management

- **Methods:** Bit Vector, Linked List, Grouping, Counting, Space Maps (e.g., ZFS).

8. Efficiency and Performance

- **Efficiency:** Pre-allocation of inodes, distributing inodes, variable cluster sizes, metadata management, dynamic table allocation.
- **Performance:** Disk controller caching, buffer cache, unified buffer cache (memory for file data and process pages), page replacement strategies, synchronous vs. asynchronous writes, sequential vs. random access optimizations (free-behind, read-ahead).

9. Mass-Storage Structure

- **Magnetic Disks:** Platters, tracks, sectors, cylinders, read-write heads.
    - **Performance Factors:** Seek time, rotational latency, transfer rate.
    - **Interfaces:** IDE/ATA, SATA, SCSI, FC.
- **Solid-State Disks (SSDs):** Use flash memory or DRAM; much faster, no moving parts.
- **Magnetic Tapes:** Primarily used for backups.
- **Disk Structure:** Mapping of Head-Sector-Cylinder (HSC) to linear block addresses. Constant Linear Velocity (CLV) vs. Constant Angular Velocity (CAV).
- **Disk Attachment:**

- **Host-Attached Storage (HAS):** Local disks connected via I/O ports (IDE, SATA, SCSI).
- **Network-Attached Storage (NAS):** Storage devices connected via network protocols (NFS, iSCSI).
- **Storage-Area Network (SAN):** Dedicated network for storage devices.

10. Disk Scheduling

- **Goal:** Minimize seek time and rotational latency to improve disk bandwidth and access time.
- **Algorithms:**
  - **FCFS (First-Come, First-Served):** Simple, fair, but inefficient.
  - **SSTF (Shortest Seek Time First):** More efficient, but can lead to starvation.
  - **SCAN (Elevator Algorithm):** Moves head back and forth across the disk, servicing requests.
  - **C-SCAN (Circular SCAN):** Moves head in one direction, then jumps back to the beginning without servicing requests on the return trip.
  - **LOOK:** Similar to SCAN, but moves only to the last pending request in each direction.
  - **C-LOOK:** Circular version of LOOK.
- **Selection:** Low load (all algorithms similar), moderate load (SSTF better), heavy load (SCAN/LOOK avoid starvation). OS priorities and disk controller scheduling can also play a role.

UNIT – I (Continued): Linux File System & Utilities

(Note: This section is a continuation of Unit I, as per the original document structure)

1. Linux File System (Detailed)

- **Directory Structure:** Root (**/**), **/bin**, **/sbin**, **/etc**, **/dev**, **/proc**, **/var**, **/tmp**, **/usr**, **/home**, **/boot**, **/lib**, **/opt**, **/mnt**, **/media**, **/srv**.
- **File Types:** Regular, Directories, Special Files.

- **File Handling Utilities: ls**, **touch**, **cat**, **cp**, **mv**, **rm**.

2. Linux Utilities (Detailed)

- **Process Utilities: ps**, **pidof**, **kill**. Foreground vs. Background processes.
- **Networking**
  Commands: ping, **host**, **finger**, **traceroute**, **netstat**, **tracepath**, **dig**, **hostname**, **route**, **nslookup**.
- **Filters: cat**, **head**, **tail**, **sort**, **uniq**, **wc**, **grep**, **tac**, **sed**, **nl**.

UNIT – II (Continued): Shell Programming & Process Scheduling

(Note: This section is a continuation of Unit II, as per the original document structure)

1. Shell Programming (Detailed)

- **Shell Basics:** Interpreter, programming language.
- **Key Concepts:** Redirection, Pipes, Variables, Metacharacters, Control Structures (**if**, **for**, **while**, **case**).
- **Scripting:** Creating and executing shell scripts.

2. Process Scheduling (Detailed)

- **Process Concepts:** Definition, states, PCB, Threads.
- **CPU Scheduling:** Objectives, queues, schedulers (long-term, short-term, medium-term), context switching.
- **Scheduling Algorithms:** FCFS, SJF, SRTF, Round Robin, Priority Scheduling.
- **Multiple-Processor Scheduling:** Load balancing, processor affinity.